Laboratório 4 de CSI-28

Descrição do Controle de Configuração – Estoque CASD

Grupo: Arthur de Sousa Vianna, Danilo Carlos da Silva, João Victor Canedo Paranhos, Matheus Lima Reis
Turma: COMP28 **Data:** 15/10/2026

---

1 Ferramentas de Configuração

Com o objetivo de gerir a configuração do projeto Estoque CASD, são usadas as seguintes ferramentas:

- git: ferramenta *open-source* de controle de versionamento do código-fonte;
- uv: pacote do Python que automatiza o controle de dependências (ex.: *framework* Django) do projeto, padronizando as versões dos pacotes utilizados entre os desenvolvedores;
- Arquivos locais de configuração (`.env`, `.my_pgpass` e `.pg_service.conf`): arquivos de texto de configuração do ambiente e do banco de dados.

2 Descrição do Controle de Configuração

2.1 Configuração Inicial

A configuração inicial do projeto requer a instalação de git, Python (versão 3.14), pacote uv e PostgreSQL (versão 18.6) na máquina de desenvolvimento.

O primeiro passo consiste em clonar o repositório GitHub do *link* `https://github.com/matlreis/Estoque-CASD` em um diretório na máquina local. Em seguida, o comando `uv sync` é executado no diretório de trabalho para sincronizar as dependências do projeto.

Utilizou-se o *framework* Django para confecção do *website*, que utiliza uma chave-secreta local e intransferível para gerar chaves em métodos internos do pacote. Essa chave-secreta pode ser gerada pelo trecho de código da Listagem 1. A chave impressa no console é inserida no arquivo `.env`, conforme a Listagem 2.

```
uv run python -c 'from django.core.management.utils import get_random_secret_key;
    print(get_random_secret_key())'
```

Listagem 1: Geração da chave-secreta do Django.

```
DJANGO_SECRET_KEY="chave-secreta-gerada"
```

Listagem 2: Arquivo `.env`.

Em seguida, é necessário configurar o banco de dados PostgreSQL para devida integração com Django. Para isso, recomenda-se a criação de um banco de dados controlada por um novo usuário. Esse procedimento pode ser feito por aplicativos que gerenciam serviços de banco de dados PostgreSQL, como o pgAdmin ou o DBeaver, ou pela própria interface de comando do PostgreSQL, como demonstrado na Listagem 3. O nome do usuário e do banco de dados são definidos a critério do desenvolvedor.

```
CREATE USER estoquecasd WITH PASSWORD 'insira_sua_senha';
CREATE DATABASE estoquecasd_db OWNER estoquecasd;
GRANT ALL PRIVILEGES ON DATABASE estoquecasd_db TO estoquecasd;
\c estoquecasd_db
GRANT ALL ON SCHEMA public TO estoquecasd;
```

Listagem 3: Criação de usuário e do banco de dados pelo PostgreSQL.

Logo depois, criam-se os arquivos `.my_pgpass` e `.pg_service.conf` conforme as Listagens 4 e 5 no diretório raiz do projeto (o mesmo que contém o arquivo `manage.py`), utilizando o nome de usuário, do banco de dados e a senha criados na etapa anterior. Importante notar que os arquivos locais de configuração do ambiente são ignorados no versionamento do código, pois são intransferíveis e apenas o respectivo desenvolvedor os acessa.

```
[my_service]
host=localhost
user=estoquecasd
dbname=estoquecasd_db
port=5432
```

Listagem 4: Arquivo `.pg_service.conf`.

```
localhost:5432:estoquecasd_db:estoquecasd:sua_senha
```

Listagem 5: Arquivo `.my_pgpass`.

2.2 Fluxo de Desenvolvimento

No contexto de desenvolvimento, o *website* é acessado pelo *localhost* da máquina local do desenvolvedor. Esse procedimento é feito por meio dos comandos da Listagem 6. O segundo e o terceiro comando refere-se à sincronização do banco de dados com os modelos criados no código-fonte; o último, por sua vez, é responsável por expor o servidor no *localhost*, podendo ser acessado na porta especificada. Esse procedimento é realizado para verificar como as alterações feitas no código se refletem no *website*.

```
uv run manage.py check # verificar se a configuração do ambiente está correta
uv run manage.py makemigrations
uv run manage.py migrate
uv run manage.py runserver
```

Listagem 6: Comandos para implementar o esquema do banco de dados e expor o *website* no *localhost*.

Cada desenvolvedor implementa um recurso em uma *branch* do projeto principal, evitando conflitos entre desenvolvedores. O histórico de modificações do projeto é mantido pela ferramenta de controle de versionamento git em um repositório centralizado, no qual os desenvolvedores realizam *push* e *pull* para enviar alterações e se atualizar com o repositório remoto, respectivamente.

Quando um recurso necessita de um novo pacote do Python, o gerenciador de pacotes uv lida automaticamente com o controle das versões e das dependências do pacote adicionado, permitindo que todos os desenvolvedores compartilhem o mesmo ambiente de desenvolvimento. Ainda, a documentação do projeto é realizada no arquivo `README.md` e as implementações são discutidas internamente entre os desenvolvedores.